

Da comunicação à transferência de arquivos

Guia de estudo e apresentação. Explica o comportamento da implementação atual, sem reproduzir funções ou código-fonte.

No Checkpoint 1, a base era fazer processos trocarem mensagens TCP corretamente. No Checkpoint 2, essa base passa a transportar documentos: dividir, identificar, comprimir, enviar, armazenar, localizar e reconstruir o conteúdo.

O que cabe à parte do Aluno 1

Upload e download, armazenamento local, fragmentação em chunks, integridade do conteúdo, compressão LZ4 e transferência paralela. Esses componentes usam a rede e o protocolo do C1 e integram-se à identidade, à tabela de membros e ao índice de metadados do projeto.

Papel	O que faz
Peer	Armazena chunks e atende pedidos. Também inicia upload e download, atuando como cliente de outro nó.
Super Peer	Mantém metadados e localizações. Informa quais Peers possuem as partes, mas não guarda os bytes dos PDFs.
CLI local	Recebe o comando digitado e o encaminha ao Peer ativo por um socket Unix local.

Por que usar o Peer ativo?

O serviço já fez JOIN e possui um NodeID conhecido. A CLI entrega a ele a operação e recebe progresso e resultado. Assim, LOOKUP e pedidos de chunks usam a identidade do serviço, e não a de um cliente anônimo criado a cada comando.

O projeto produz bin/peer e bin/superpeer, com entradas em peer.c e superpeer.c. Os nomes bin/client e bin/node são apenas aliases de compatibilidade. Um mesmo Peer pode servir um documento e solicitar outro: cliente/servidor descreve o papel em uma interação, não uma divisão permanente entre máquinas.

Frase central: o Super Peer informa onde buscar; os Peers guardam e transferem o conteúdo diretamente entre si.

Chunks, identificadores e integridade

Antes de acompanhar uma transferência, separe o arquivo, suas partes e os identificadores usados pelo protocolo.

Fragmentação em partes de até 4 MiB

Nesta implementação, cada chunk contém até 4.194.304 bytes originais. Um arquivo de 10 MiB gera três partes. Elas não precisam ser PDFs válidos isoladamente: são regiões do arquivo original.

Chunk	Offset inicial	Tamanho original
0	0 bytes	4 MiB
1	4.194.304 bytes	4 MiB
2	8.388.608 bytes	2 MiB

O **índice** identifica a parte; o **offset** indica onde seus bytes pertencem no documento. Mesmo que o chunk 2 chegue antes do 0, é possível gravá-lo na posição correta. O último chunk pode ser menor que os demais.

Três identificadores, três finalidades

NodeID: identifica o nó, a partir de IP, porta e UUID. **ObjectID**: é o SHA-256 dos bytes do arquivo inteiro. **TransactionID**: relaciona um pedido com sua resposta. NodeID e ObjectID têm 32 bytes; TransactionID tem 16. Exibir um hash de 32 bytes em hexadecimal exige 64 caracteres.

CRC32 não substitui SHA-256

O CRC32, calculado pela zlib, verifica a mensagem serializada: header com checksum zerado, seguido do payload. O SHA-256, fornecido pelo OpenSSL, verifica os bytes originais do chunk e do documento. São verificações em níveis diferentes; nenhuma delas, sozinha, autentica o remetente.

O hash do documento é calculado incrementalmente, lendo blocos e atualizando o estado do SHA-256. Não é necessário carregar o PDF inteiro na memória. Ele também **não** é obtido concatenando os hashes dos chunks.

Exemplo importante: dois arquivos com nomes diferentes, mas bytes idênticos, têm o mesmo ObjectID. O modelo atual preserva o primeiro nome cadastrado; outro nome não se torna automaticamente um alias de consulta.

Upload: preparar, enviar e confirmar

file_client.c coordena o envio; peer_service.c recebe os pedidos; storage.c verifica e persiste o conteúdo.

1. Preparar o documento

O Peer executor verifica a extensão .pdf, sem diferenciar maiúsculas e minúsculas, abre o arquivo e calcula tamanho e ObjectID. Essa política não faz validação estrutural de PDF. O arquivo de origem não deve ser alterado durante a operação.

2. BEGIN: declarar o que será enviado

O executor descobre a identidade do Peer de armazenamento e envia os metadados: ObjectID, nome, tamanho, quantidade de chunks e compressão. O receptor prepara ou recupera um upload compatível em pending. BEGIN ainda não significa que o documento está completo.

3. CHUNK: enviar partes independentes

Workers pegam índices de trabalho, leem cada região com pread, calculam SHA-256 dos bytes originais e comprimem com LZ4. O pedido leva ObjectID, índice, offset, tamanhos original/comprimido, hash e conteúdo LZ4. Cada worker usa sua própria conexão TCP.

LZ4 é aplicada individualmente: não existe um único fluxo comprimido dependente de todos os chunks anteriores. A capacidade de saída considera o pior caso da biblioteca, pois comprimir pode não reduzir o tamanho, especialmente em PDFs já comprimidos.

4. Verificar e armazenar cada parte

O receptor primeiro passa pela validação de framing e CRC. Depois confere o descritor, descomprime temporariamente e calcula o hash do conteúdo original. Só aceita o chunk se tamanhos, posição e integridade forem compatíveis com o documento. No disco ficam os bytes comprimidos e o descritor no manifest.

5. COMMIT e anúncio

Depois de aguardar os workers, o executor pede COMMIT. O receptor exige todas as partes, lê e descomprime em ordem e recalcula o SHA-256 completo. Após validar tamanho e ObjectID, publica o documento e envia ANNOUNCE ao Super Peer. O índice registra descritores e localização do Peer.

Quando há sucesso? Somente após a confirmação do registro. Se o anúncio falhar, os bytes podem estar válidos no Peer e o upload ainda retornar erro. Essa sequência não é uma transação distribuída 2PC.

Persistência: o que fica no disco

storage.c distingue uma parte recebida, um documento completo e um objeto publicado para leitura.

Elemento	Finalidade
pending	Uploads ainda não publicados. Permite reutilizar chunks compatíveis numa nova tentativa.
objects	Documentos publicados após verificação completa. É a área usada para atender downloads.
manifest.bin	Registro versionado com metadados, dono, estado e descritores de cada chunk.
node.uuid	Componente persistente da identidade do Peer; com IP e porta estáveis, mantém o NodeID.

O manifest é um inventário, não outro PDF

Ele registra nome, ObjectID, tamanho, compressão, quantidade de chunks, NodeID proprietário e informações das partes: índice, offset, tamanhos, hash e presença. Os diretórios internos usam ObjectID hexadecimal; o nome recebido não é usado para montar o caminho físico dos chunks.

Por que temporário, fsync e publicação?

Gravar bytes não significa que o documento está pronto. O código escreve temporários, solicita sincronização com fsync e publica por mudança de nome. Também sincroniza diretórios para persistir as alterações de nomes. A intenção é não expor como concluído um estado que ainda está em preparação.

Se persistir um manifest de chunk falhar, o código restaura o descritor e estado anteriores em memória. No COMMIT, verifica todos os chunks antes de confirmar FINISHED. Isso protege contra falso sucesso, mas não significa que qualquer falha de disco possa ser revertida sem condições.

O que acontece ao reiniciar?

O Peer carrega os manifests válidos. Para objetos finalizados, verifica os chunks e o hash do documento; depois de JOIN, anuncia novamente seu catálogo ao Super Peer. Uploads pendentes compatíveis podem continuar quando o usuário repete a operação.

O índice do Super Peer é volátil. Reiniciar o Super Peer e depois os Peers reconstrói as localizações pelos anúncios. Reiniciar somente o Super Peer não faz os Peers ativos reanunciarem automaticamente nesta implementação.

Idempotência: repetir o upload do mesmo ObjectID com dados compatíveis não deve criar outro objeto nem aceitar uma parte conflitante. Retomada de pending não é recuperação automática de todas as falhas distribuídas.

Download: localizar e reconstruir

O destino recebe apenas um arquivo confirmado após as verificações. O conteúdo não passa pelo Super Peer.

1. LOOKUP no Super Peer

O Peer executor consulta por nome ou ObjectID. `directory.c` combina `metadata.c`, que conhece documentos e chunks, com `membership.c`, que conhece NodeIDs e endpoints. A resposta contém metadados, descritores e uma lista de Peers para cada parte. Um nome associado a ObjectIDs diferentes é ambíguo: deve-se usar o ID.

2. Criar a saída parcial

`file_client.c` cria um arquivo `.part` com exclusividade. Se o destino ou um `.part` preexistente impedir a operação, não toma posse desses arquivos para sobrescrevê-los. A saída é dimensionada para receber os chunks nas posições esperadas.

3. Buscar os chunks diretamente

Os workers enviam `DOWNLOAD_REQ` aos Peers anunciados. O Peer de armazenamento lê o chunk em `storage.c` e devolve `DOWNLOAD_REP`. O executor confere o descritor recebido contra o esperado, descomprime, verifica SHA-256 e usa `pwrite` no offset correto. A ordem de chegada não precisa coincidir com a ordem do documento.

4. Tentar outra localização quando necessário

Se um endpoint falhar ou não responder dentro do prazo, o worker tenta o próximo anunciado para aquela parte. Isso é fallback entre cópias existentes. Não cria novas cópias e não é replicação automática. Sem localização utilizável, a operação deve retornar erro.

5. Verificação final e publicação

Após aguardar todos os workers, o programa sincroniza o `.part` e calcula seu tamanho e ObjectID. Se corresponderem ao documento esperado, usa link para criar o nome final sem substituir um destino existente, sincroniza o diretório e remove o nome temporário.

O uso de link evita sobrescrita até se outro processo criar o destino após uma checagem anterior. Ainda assim, uma falha de sincronização depois da publicação pode deixar o destino existente e retornar erro: visibilidade atômica não é rollback garantido de toda a operação.

Como explicar: cada parte é validada antes de ser gravada, e o documento inteiro é validado novamente antes de confirmar o download. Isso verifica tanto o conteúdo das partes quanto a reconstrução final.

Paralelismo e transporte seguro

São duas formas diferentes de concorrência: processar chunks e atender conexões.

Workers de uma transferência

Um pool POSIX distribui os índices de chunks. O padrão é o menor valor entre CPUs disponíveis, quantidade de chunks e 8. `PEER_TRANSFER_THREADS` permite configurar de 1 a 32, ainda limitado ao trabalho disponível. A variável deve estar no ambiente do Peer executor que realiza a operação.

Mutexes protegem o próximo índice, os contadores e o primeiro erro. Eles não precisam envolver toda a transmissão. Ao ocorrer falha, para-se de distribuir novas tarefas, mas as threads iniciadas são aguardadas. `pread` e `pwrite` usam posição explícita e não disputam o cursor compartilhado do arquivo.

Workers do servidor

`concurrent_server.c` tem 32 workers e fila de até 64 conexões. O mesmo runtime serve Peer e Super Peer, com handlers diferentes. Isso limita recursos, em vez de criar indefinidamente uma thread para cada chegada. O controle Unix aceita um comando local por vez, mas seus chunks continuam paralelos.

TCP entrega um fluxo, não mensagens prontas

Uma chamada `send` ou `recv` pode tratar apenas parte do que foi pedido. `network.c` repete a operação e acompanha a quantidade acumulada. O protocolo lê um header de 98 bytes e, depois, exatamente o `payload_size` indicado. Esse framing permite separar mensagens no fluxo TCP.

O payload tem limite de 5 MiB para comportar um chunk de 4 MiB, inclusive a possível expansão LZ4 e seu descritor. Campos inteiros são serializados em ordem de rede: não se transmite uma struct C bruta, que poderia conter padding e ponteiros.

Prazos e correlação

A rede usa, por padrão, 5 segundos para conexão e 30 segundos de inatividade de I/O. Havendo progresso, o prazo de inatividade é renovado. Não é um prazo total de 30 segundos para o PDF inteiro. RPC confere `TransactionID` e identidades esperadas; a camada da operação confere o tipo da resposta.

Não confundir: ter vários workers configurados não prova execução simultânea; CRC correto não prova autoria; fallback não prova detecção automática de falhas.

Mapa dos componentes e integração

Use este mapa para explicar responsabilidades sem precisar decorar cada função.

Arquivo ou conjunto	Responsabilidade no fluxo
peer.c + local_control.c	Receber o comando e encaminhá-lo ao serviço Peer com identidade ativa.
peer_service.c	Inicializar o armazenamento, fazer JOIN, atender STORE/DOWNLOAD e anunciar documentos.
file_client.c	Coordenar upload/download, distribuir chunks, validar o resultado e informar progresso.
compression.c + content.c	Encapsular LZ4 e auxiliares de hash, extensão, nomes e sincronização.
storage.c	Verificar chunks, persistir manifests, publicar objetos e recuperar o catálogo local.
transfer_protocol.c	Codificar metadados, chunks, consultas e respostas em payloads de bytes.
rpc.c + protocol.c + network.c	Trocar requisição/resposta, delimitar/verificar mensagens e transportar bytes TCP.
concurrent_server.c	Distribuir conexões entre workers com limites de recursos.
node.c + app_config.c	Fornecer identidade e configuração utilizadas pelos serviços.
superpeer_app.c + directory.c	Receber anúncios/consultas e ligar os handlers ao catálogo e aos membros.
metadata.c + membership.c	Manter o modelo de documentos/localizações e a tabela de nós. Integração com a parte do Aluno 2.

Onde ocorre a passagem entre as partes dos alunos?

Após o Peer confirmar um documento, envia ANNOUNCE com seus metadados e descritores. O Super Peer valida o membro e registra o conjunto por metadata_announce. No LOOKUP, devolve esses descritores e resolve NodeIDs em IP/porta. O Aluno 1 usa esse contrato para transferir os bytes; não precisa implementar o índice dentro do transporte.

Os headers .h expõem esses contratos; os .c implementam os comportamentos. Um auxiliar estar em metadata.c, como o cálculo incremental do ObjectID, não muda o fato de que o fluxo de upload precisa utilizá-lo.

Limite do C2: não apresentar Chord, Gossip, eleição, replicação automática, 2PC ou disponibilidade de 99,5% como resultados concluídos. Reservar códigos de mensagem ou estados não implementa seus algoritmos.

Roteiro para explicar e demonstrar

Apresente primeiro o caminho dos dados. Depois explique por que existem hashes, chunks, compressão e controle de estado.

Uma fala de aproximadamente dois minutos

“No Checkpoint 2, minha parte usa a rede do primeiro checkpoint para transferir documentos. O Peer ativo recebe a operação, calcula o SHA-256 do arquivo inteiro como ObjectID e divide o conteúdo em chunks de até 4 MiB. Cada parte recebe seu próprio hash, é comprimida com LZ4 e pode ser enviada em paralelo.”

“O receptor valida mensagens e conteúdo, salva chunks e manifest em pending e só publica o documento depois de conferir todas as partes e o hash final. Em seguida anuncia sua disponibilidade ao Super Peer. No download, outro Peer consulta esse índice, busca os chunks diretamente nos Peers, verifica e remonta o arquivo. O Super Peer guarda localizações, não os bytes do PDF.”

Demonstração sugerida, em terminais separados

1. Inicie Super Peer em 55101, Peer A em 55102 e Peer B em 55103. Espere o JOIN dos dois.
2. Faça upload de um PDF maior que 4 MiB para A e mostre ObjectID, número de chunks, hashes e compressão.
3. Faça download usando B como executor: selecione --local-peer-port 55103. O endpoint remoto da consulta continua sendo o Super Peer.
4. Compare original e destino com cmp e mostre a confirmação SHA-256.
5. Explique o manifest e mostre que chunks ficam no Peer, não no Super Peer.

Perguntas que você deve conseguir responder

Por que verificar o arquivo inteiro se cada chunk já foi verificado? Para conferir a identidade final e a reconstrução do documento, incluindo tamanho e sequência completa.

Se o chunk 2 chegar antes do 0? O offset permite gravá-lo na posição correta.

Por que LZ4 por chunk? Para processar cada parte independentemente, sem depender de um fluxo comprimido anterior.

Por que um upload repetido pode manter outro nome? A identidade é o conteúdo; o modelo atual preserva o primeiro nome do objeto.

Cuidados antes de mostrar os testes

Use portas livres e não deixe servidores manuais concorrendo com a suíte. Uma cópia com outro nome, mas mesmo ObjectID, pode falhar numa consulta por esse novo nome. Diferencie esse contrato de um problema de integridade. Este material não executou novamente os testes e não atribui aprovação atual a resultados históricos.

Base de leitura: fontes do projeto, guia_resumido_arquivos.md e integracao_checkpoint_2_aluno_2.md. Explicação da implementação consultada em 29/09/2026; trechos históricos de outros documentos podem descrever versões anteriores.